REST APIs, Hono & Kubb

PB138 — Basics of Web Development

"Your frontend and backend finally need to talk. Let's teach them how."

Agenda

1. HTTP — the language of the web
2. Client-Server lifecycle
3. REST API design
4. Building APIs in TypeScript — Express → tsoa → Hono → Hono + zod-openapi
5. CORS
6. OpenAPI specification
7. React Hooks & TanStack Query
8. Kubb — code generation from specs

HTTP

The protocol that powers the internet — built in 1989, still held together with duct tape and optimism

What is HTTP?

- **H**yper**T**ext **T**ransfer **P**rotocol
- A **request-response** protocol: client asks, server answers
- **Stateless**: every request is independent, server remembers nothing
- Text-based (HTTP/1.1), then binary (HTTP/2, HTTP/3)
- Runs over TCP, usually port `80` (HTTP) or `443` (HTTPS)

```
Client                                Server
 │                                          │
 │   GET /api/users HTTP/1.1                │
 │  ──────────────────────────────────────>│
 │                                          │
 │   HTTP/1.1 200 OK                        │
 │   [{"id":1,"name":"Alice"}]              │
 │  <──────────────────────────────────────│
```

HTTP Message Anatomy

```
Request:
GET /api/users?role=admin HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGci...
Content-Type: application/json

{"name": "Alice", "email": "a@b.com"}
```

```
Response:
HTTP/1.1 200 OK
Content-Type: application/json
Location: /api/users/42

{"id": 42, "name": "Alice"}
```

- **Method + Path** — what to do and to which resource
- **Headers** — metadata (auth, content type, caching, CORS...)
- **Body** — data payload (request: POST/PUT/PATCH; response: usually JSON)
- **Query params** — filtering, sorting, pagination (`?role=admin`)
- **Status code** — how it went (in response)

HTTP Methods

Method	Purpose	Has Body?	Idempotent?
`GET`	Read a resource	No	Yes
`POST`	Create a resource	Yes	No
`PUT`	Replace a resource fully	Yes	Yes
`PATCH`	Partially update	Yes	Usually
`DELETE`	Remove a resource	No	Yes

Idempotent = calling it N times gives the same result as calling it once.

`DELETE /users/1` twice → same state. `POST /users` twice → two users.

HTTP Status Codes

Range	Category	Common codes
2xx	Success	`200 OK`, `201 Created`, `204 No Content`
3xx	Redirect	`301 Moved Permanently`, `304 Not Modified`
4xx	Client error	`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `409 Conflict`, `422 Unprocessable Entity`
5xx	Server error	`500 Internal Server Error`, `503 Service Unavailable`

> *"Is it a 401 or 403?"*
>
> `401` = who are you? (not logged in)
>
> `403` = I know who you are, you just can't do this

Status Codes in Practice

```
// Correct usage
res.status(201).json(newUser)                // POST — resource created
res.status(200).json(users)                  // GET  — here's your data
res.status(204).send()                       // DELETE — done, nothing to return
res.status(404).json({ error: "User not found" })
res.status(400).json({ error: "Invalid input" })
res.status(401).json({ error: "Login required" })
res.status(403).json({ error: "Admins only" })

// What juniors (and some seniors) do
res.status(200).json({ error: "User not found" })  // wrong
res.status(200).json({ success: false })            // wrong
```

Status codes ARE part of your API contract. Clients branch on them.

Client-Server Lifecycle

What actually happens between `fetch()` *and* `.json()`

The Full Journey

```
React App                                Server
 │                                                │
 │   1. fetch('/api/products')                    │
 │      DNS: resolve hostname → IP                │
 │      TCP: 3-way handshake                      │
 │      TLS: negotiate if HTTPS                   │
 │                                                │
 │   2. HTTP Request ──────────────────────────>  │
 │      GET /api/products                         │   3. Router matches path
 │      Authorization: Bearer xyz                 │   4. Middlewares run (auth, log...)
 │                                                │   5. Route handler executes
 │                                                │   6. Data fetched (DB / memory)
 │   7. HTTP Response <──────────────────────────│
 │      200 OK                                    │
 │      [{"id":1,"name":"Widget"}]                │
 │                                                │
 │   8. React updates state → re-render           │
```

REST API

Constraints you'll hate — until you work with an API that ignores them all

What is REST?

REpresentational **S**tate **T**ransfer — an architectural style, not a protocol.

Defined by Roy Fielding in his **2000 PhD dissertation**. Yes, really.

Core constraints:

- **Stateless** — server stores no session; each request is self-contained
- **Uniform interface** — resources identified by URLs, standard HTTP methods
- **Client-server** — clear separation of concerns
- **Cacheable** — responses indicate whether they can be cached
- **Layered** — client doesn't know if it talks to a proxy or the real server

Resource-Oriented URLs

Think in **nouns**, not verbs.

```
Verb-based (not REST)          Resource-based (REST)

/getUsers                      GET     /api/users
/createUser                    POST    /api/users
/updateUser?id=1               PUT     /api/users/1
/deleteUser?id=1               DELETE  /api/users/1
/getUserPosts?userId=1         GET     /api/users/1/posts
```

The URL identifies **what**.

The HTTP method identifies **what to do with it**.

Richardson Maturity Model

A way to score how "RESTful" an API actually is.

```
Level 3 │  HATEOAS ─── responses include links to next actions
        │
Level 2 │  HTTP Verbs + Status Codes ─── use methods correctly
        │                              ▲ most real-world APIs live here
Level 1 │  Resources ─── separate URL per resource
        │
Level 0 │  Single Endpoint ─── one URL, everything is POST
```

Target: Level 2 — correct HTTP methods + status codes. Browsers and CDNs can cache `GET` automatically. `DELETE` is idempotent. Clients branch on status codes, not body parsing.

Building APIs in TypeScript

From Express to Hono — how the ecosystem evolved

Express.js

"Node.js framework so minimal, you'll add 40 packages before writing a single route"

What is Express?

- Minimal, unopinionated web framework for Node.js
- Created in 2010, still the most-used Node.js backend framework
- `npm install express` → you have a server

```
import express from 'express'

const app = express()
app.use(express.json()) // parse JSON request bodies

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' })
})

app.listen(3000, () => {
  console.log('Server on http://localhost:3000')
})
```

That's it. That's a server.

The Middleware Chain

Every request flows through a pipeline of functions.

```
Incoming Request
      |
      v
  express.json()    -- parse request body
      |
      v
  cors()            -- set CORS headers
      |
      v
  authStub()        -- attach currentUser
      |
      v
  Route Handler     -- app.get('/api/users', handler)
      |
      v
  Response sent
```

Middleware — The Anatomy

A middleware is just a function: `(req, res, next) => void`

```
// Logging middleware
const logger = (req, res, next) => {
  console.log(`→ ${req.method} ${req.path}`)
  next() // must call next() or the request hangs forever
}

// Auth stub — Week 8 replaces this with real OAuth
const authMiddleware = (req, res, next) => {
  req.currentUser = { id: '1', role: 'admin' }
  next()
}

app.use(logger)
app.use(authMiddleware)
```

`next()` = "pass control to the next function in the chain."

Forgetting it = request hangs silently. Classic Friday afternoon bug.

Route Handlers

```
const products: Product[] = [] // in-memory for now — Week 5 adds a real DB

// GET all
app.get('/api/products', (req, res) => {
  res.json(products)
})

// GET one
app.get('/api/products/:id', (req, res) => {
    const product = products.find(p => p.id === req.params.id)
    if (!product) return res.status(404).json({ error: 'Not found' })
    res.json(product)
})

// POST — create
app.post('/api/products', async (req, res) => {
  const product = { id: crypto.randomUUID(), ...req.body }
  products.push(product)
  res.status(201).json(product)
})
```

Runtime Validation with Zod

Remember Week 3? `req.body` is `any`. Let's fix that.

```
import { z } from 'zod'

const CreateProductSchema = z.object({
  name: z.string().min(1),
  price: z.number().positive(),
  category: z.enum(['electronics', 'clothing', 'food']),
})

app.post('/api/products', async (req, res) => {
  const result = CreateProductSchema.safeParse(req.body)
  if (!result.success) {
    return res.status(400).json({ error: result.error.flatten() })
  }
  // result.data is fully typed
  const product = { id: crypto.randomUUID(), ...result.data }
  products.push(product)
  res.status(201).json(product)
})
```

tsoa — Decorators Instead of Manual Routes

Instead of writing `app.get(...)` by hand, we decorate controller classes:

```
Developer writes            tsoa generates
----------------           ----------------
@Route, @Get, @Post         src/generated/routes.ts  (RegisterRoutes)
on controller classes       openapi.json             (API spec)
```

```
// app.ts — just one line to register everything
import { RegisterRoutes } from './generated/routes'

const app = express()
app.use(express.json())
app.use(cors())
RegisterRoutes(app)   // all routes from all controllers
```

You write controllers with decorators → run `bun run generate` → tsoa creates routes + OpenAPI spec.

Docs: tsoa-community.github.io/docs

tsoa Controller Example

```
import { Controller, Route, Tags, Get, Post, Query, Path, Body } from 'tsoa'

@Route('courses')
@Tags('Courses')
export class CoursesController extends Controller {

  @Get()
  public async getCourses(
    @Query() semester?: 'fall' | 'spring',
    @Query() minCredits?: number,
  ): Promise<Course[]> {
    return coursesService.getAll({ semester, minCredits })
  }

  @Get('{id}')
  public async getCourseById(@Path() id: string): Promise<Course> {
    const course = coursesService.getById(id)
    if (!course) { this.setStatus(404); throw new Error('Not found') }
    return course
  }

  @Post()
  public async createCourse(@Body() body: CreateCourseBody): Promise<Course> {
    this.setStatus(201)
    return coursesService.create(body) // Zod validates in the service
  }
}
```

Hono

"Fast, lightweight, built for the edge — and it just works with Bun"

What is Hono?

- Modern, lightweight web framework for TypeScript
- **Works everywhere** — Bun, Deno, Node.js, Cloudflare Workers, Vercel
- Similar API to Express, but with native TypeScript support
- Built-in middleware (CORS, logger, etc.) — no extra packages

```
import { Hono } from 'hono'

const app = new Hono()

app.get('/api/health', (c) => {
  return c.json({ status: 'ok' })
})

export default { port: 3000, fetch: app.fetch } // Bun native server
```

No `express.json()`, no `app.listen()` — Hono handles it.

Hono Route Handlers

```
const products: Product[] = []

// GET all
app.get('/api/products', (c) => {
  return c.json(products)
})

// GET one
app.get('/api/products/:id', (c) => {
  const product = products.find(p => p.id === c.req.param('id'))
  if (!product) return c.json({ error: 'Not found' }, 404)
  return c.json(product)
})

// POST — create
app.post('/api/products', async (c) => {
  const body = await c.req.json()
  const product = { id: crypto.randomUUID(), ...body }
  products.push(product)
  return c.json(product, 201)
})
```

Same concepts as Express. Different API — `c.json()` instead of `res.json()`.

@hono/zod-openapi

"One Zod schema to validate, type, and document them all"

The Problem with Separate Concerns

With Express + tsoa, you write things three times:

```
TypeScript interface    →  defines the shape
Zod schema              →  validates the input
tsoa decorators         →  generates the OpenAPI spec
```

Each one can go out of sync. `@hono/zod-openapi` unifies all three.

@hono/zod-openapi — Single Source of Truth

One Zod schema = types + validation + OpenAPI spec generation:

```
import { z } from '@hono/zod-openapi'

export const CreateProductSchema = z.object({
  name: z.string().min(1).max(100),
  price: z.number().positive(),
  category: z.enum(['electronics', 'clothing', 'food']),
}).openapi('CreateProduct')

// TypeScript type is inferred — no separate interface needed
type CreateProduct = z.infer<typeof CreateProductSchema>
```

`.openapi('CreateProduct')` registers the schema for the OpenAPI spec.

Route Definitions

Routes are defined declaratively — method, path, schemas, responses:

```
import { createRoute, OpenAPIHono, z } from '@hono/zod-openapi'

const app = new OpenAPIHono()

app.openapi(
  createRoute({
    method: 'get',
    path: '/',
    responses: {
      200: {
        description: 'List of products',
        content: { 'application/json': { schema: z.array(ProductSchema) } },
      },
    },
  }),
  (c) => {
    return c.json(products, 200) // fully typed — TS knows the shape
  },
)
```

Request validation happens automatically — invalid input returns 400 before your handler

The Evolution — At a Glance

	Express	Express + tsoa	Hono + zod-openapi
Language	JS / TS	TS	TS
Types	manual	decorators	inferred from Zod
Validation	manual Zod	manual Zod	automatic
OpenAPI	manual / tsoa gen	tsoa gen	built-in
Code gen needed	—	`tsoa generate`	—
Runtime	Node.js	Node.js	Bun, Node, Edge

Our assignment uses Hono + @hono/zod-openapi — the rightmost column.

CORS

"The browser's way of saying: I don't know you like that"

Same-Origin Policy

Browsers block requests to different origins by default.

Origin = protocol + hostname + port

```
http://localhost:5173     ← your React app (Vite)
http://localhost:3000     ← your Express API

Different ports → different origins → browser blocks it
```

This is a **security feature**, not a bug.

Imagine if any website could call `https://yourbank.com/api/transfer` using your logged-in session cookies... yeah.

CORS — The Solution

Cross-**O**rigin **R**esource **S**haring — the server tells the browser which origins it trusts.

```
// Express
import cors from 'cors'
app.use(cors({ origin: ['http://localhost:5173'] }))

// Hono — built-in, no extra package
import { cors } from 'hono/cors'
app.use('*', cors({ origin: ['http://localhost:5173'] }))
```

The server adds `Access-Control-Allow-Origin` headers.
The browser checks them before showing you the response.

For non-simple requests (custom headers like `Authorization`), the browser sends an `OPTIONS` preflight first. If the server doesn't respond with the right CORS headers → the famous CORS error.

Works fine in Postman but breaks in the browser? It's always CORS.

Checkpoint: CORS

- Same-Origin Policy: browsers block cross-origin requests by default
- CORS lets your server declare which origins it trusts
- Never use `origin: '*'` in production (unless it's a truly public API)
- Set `credentials: true` if using cookies or Authorization headers
- CORS errors = server didn't include the right response headers

Questions?

OpenAPI

"Because 'just read the source code' is not documentation"

What is OpenAPI?

A standard, machine-readable description of your HTTP API.

One spec, many superpowers:

- Interactive docs (Swagger UI, Redoc)
- Typed client generation — Kubb, openapi-typescript
- Request/response validation
- Contract between frontend and backend teams
- Mock servers for testing before the backend exists

Previously known as **Swagger**. OpenAPI 3.x is the current standard.

Schema-First vs Code-First

Two approaches to owning the spec:

```
Schema-First                    Code-First
────────────────                ────────────────
Write openapi.yaml              Write routes with Zod schemas
│                               │
▼                               ▼
Generate server stubs           @hono/zod-openapi extracts the spec
Generate client code            │
│                               ▼
▼                               Generate openapi.json
Implement the handlers
```

We use code-first — write your routes with Zod schemas, generate the spec.

More natural, stays in sync with your code automatically.

No YAML authoring by hand at 2am.

OpenAPI Spec — A Peek

```yaml
openapi: 3.0.0
info:
  title: Products API
  version: 1.0.0
paths:
  /api/products:
    get:
      summary: List all products
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Product'
    post:
      summary: Create a product
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateProduct'
```

Swagger UI — Your API Playground

Generated from `openapi.yaml` — a live, interactive UI:

- **GET** `/api/products` — list all
- **POST** `/api/products` — create
- **GET** `/api/products/{id}` — get one
- **PUT** `/api/products/{id}` — update
- **DELETE** `/api/products/{id}` — delete

Each endpoint has a **"Try it out"** button — fills in params, sends real requests, shows live responses.

Every team member can explore and test the API without reading source code.

Checkpoint: OpenAPI

- Machine-readable API description — one source of truth
- Enables docs, client generation, validation, mock servers
- Schema-first: write YAML → generate code
- Code-first: write code → generate YAML
- We use code-first — stays in sync automatically
- Swagger UI = interactive docs + live API playground

Questions?

React Hooks

The functions that make React components actually do things

Built-in Hooks — The Essentials

```
// useState — reactive state
const [count, setCount] = useState(0)
const [user, setUser] = useState<User | null>(null)

// useEffect — side effects (API calls, subscriptions, timers)
useEffect(() => {
  console.log('Component mounted')
  return () => console.log('Component unmounted') // cleanup
}, []) // [] = run once on mount

useEffect(() => {
  console.log('Count changed to', count)
}, [count]) // runs when count changes

// useMemo — expensive computations, cached until deps change
const sorted = useMemo(() => users.sort(byName), [users])

// useRef — mutable value that doesn't trigger re-render
const inputRef = useRef<HTMLInputElement>(null)
```

Rules: only call hooks at the **top level** — never inside `if`, loops, or callbacks.

Custom Hooks

Extract reusable logic into functions starting with `use`.

```
// useLocalStorage — persist state across page reloads
function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key)
    return stored ? JSON.parse(stored) : initial
  })

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value))
  }, [key, value])

  return [value, setValue] as const
}

// Usage — same API as useState, but survives refresh
const [theme, setTheme] = useLocalStorage('theme', 'light')
const [cart, setCart] = useLocalStorage<CartItem[]>('cart', [])
```

Custom hooks = composable, testable, reusable pieces of stateful logic.

TanStack Query — Data Fetching Done Right

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

function ProductList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['products'],
    queryFn: () => fetch('/api/products').then(r => r.json()),
  })

  if (isLoading) return <p>Loading...</p>
  if (error) return <p>Error: {error.message}</p>
  return <ul>{data.map(p => <li key={p.id}>{p.name}</li>)}</ul>
}
```

What you get **for free**: caching, background refetching, deduplication, retry on error, loading/error states, window focus refetching, stale-while-revalidate.

`queryKey` = cache key. Same key → same cached data across components.
`queryFn` = how to fetch. TanStack Query handles **when** and **how often**.

Kubb

Stop writing `fetch('/api/users')` *by hand like it's 2015*

Kubb — Code Generation from OpenAPI

```
Hono Routes  -->  openapi.json  -->  Generated Client
                  (kubb.config.ts)

GET    /products                        getProducts()
POST   /products                        createProduct(data)
GET    /products/:id                    getProductById(id)
PUT    /products/:id                    updateProduct(id, data)
DELETE /products/:id                    deleteProduct(id)
```

All fully typed. No manual fetch. Backend changes → TypeScript yells at you.

What Kubb Generates

Kubb generates **query options factories** — functions returning `{ queryKey, queryFn }`:

```
// Generated by Kubb — you never write this
export function getProductsQueryOptions() {
  return {
    queryKey: ['products'],
    queryFn: () => fetch('/api/products').then(r => r.json()) as Promise<Product[]>,
  }
}
```

You use them with standard `useQuery` — no magic hooks, full control:

```
import { useQuery } from '@tanstack/react-query'
import { getProductsQueryOptions } from '@/api/generated'

// Basic usage — just spread the generated options
const { data, isLoading } = useQuery(getProductsQueryOptions())

// Override anything — e.g. poll every 5s for a live dashboard
const { data } = useQuery({
  ...getProductsQueryOptions(),
  refetchInterval: 5000,
})

// Disable the query until user picks a category
const { data } = useQuery({
  ...getProductsByCategory(selectedCategory),
  enabled: selectedCategory !== null,
})
```

Same `useQuery` you already know. Kubb generates the boring parts — you keep full control.

Why Kubb

- **Type safety end-to-end** — backend types flow to the frontend automatically
- **No drift** — spec changes → regenerate → TypeScript catches all breakages
- **Zero manual fetch code** — routes, types, React Query hooks all generated
- **Supports many outputs** — Axios, fetch, React Query, SWR, Zod validators

```
# Add to package.json
"generate": "kubb generate"

# Run whenever the backend changes:
npm run generate
```

The OpenAPI spec becomes the **single source of truth**.

Frontend and backend can evolve independently — safely.

Summary

What We Covered

1. **HTTP** — message anatomy, methods, status codes
2. **Client-Server lifecycle** — DNS → TCP → middleware → handler → response
3. **REST** — stateless, resource-oriented, target Level 2
4. **Express.js** — middleware chain, route handlers, Zod validation
5. **tsoa** — decorator-based controllers, generated routes + spec
6. **Hono** — modern, lightweight, works with Bun natively
7. **@hono/zod-openapi** — single source of truth: types + validation + OpenAPI
8. **CORS** — why it exists, how to configure it correctly
9. **OpenAPI** — machine-readable spec, code-first approach
10. **React Hooks & TanStack Query** — useQuery, caching, background refetching
11. **Kubb** — typed query options generated from the OpenAPI spec

"A good API is not built once. It's maintained with your future self in mind."

Now let's build something.

Exercises time!